

CHESS Multiplayer

A Command-Line Chess Game in C++

CS112 — Object Oriented Programming & Design

Project Report by:

Haider Ali	2025245
Azan Ahmed	2025176

Course Code: CS112

Instructor: Muhammad Sajid Ali

Total Marks: 100

Deadline: 10/05/2026

GitHub: <https://github.com/Haider-3100/CS-112-Project>

Table of Contents

- 1. Introduction**
- 2. Motivation**
- 3. Purpose and Objectives**
- 4. Problem Statement**
- 5. Proposed Solution**
- 6. System Architecture**
- 7. OOP Concepts Applied**
 - 7.1 Classes and Objects
 - 7.2 Abstraction
 - 7.3 Encapsulation
 - 7.4 Inheritance
 - 7.5 Polymorphism
 - 7.6 Association / Composition
 - 7.7 Modular Design
 - 7.8 File Handling
- 8. Code Walkthrough & Snippets**
 - 8.1 Pieces.h — Piece Hierarchy
 - 8.2 Board.h — Board Class Interface
 - 8.3 Pieces.cpp — Movement Logic
 - 8.4 Board.cpp — Game Logic
 - 8.5 Display.cpp — Board Rendering
 - 8.6 main.cpp — Game Loop
- 9. How to Build and Run**
- 10. Sample Gameplay Output**
- 11. Conclusion**

1. Introduction

This report documents the design and implementation of a fully functional, command-line Chess Engine developed in C++ as the semester project for CS112 (Object Oriented Programming & Design). The project demonstrates the practical application of core OOP principles — encapsulation, inheritance, polymorphism, abstraction, and association — within the domain of a real-world strategy board game.

Chess is a two-player strategy game played on an 8×8 board with 16 pieces per side. Each piece type (King, Queen, Rook, Bishop, Knight, Pawn) follows distinct movement rules, making chess an ideal candidate for an OOP-based design: each piece is naturally modelled as a class derived from a common abstract base.

The system supports all standard gameplay mechanics including legal move filtering with check detection, checkmate and stalemate recognition, pawn promotion, and castling — features that go well beyond basic CRUD and demonstrate a simulation of real-world logic.

2. Motivation

Chess was chosen as the project domain for three primary reasons:

- **Natural OOP Mapping:** The six distinct piece types map perfectly to a class hierarchy. The abstract Piece base class defines the contract (isValidMove, getDisplay) while each subclass provides its own specialised implementation — a textbook application of polymorphism and abstraction.
- **Non-trivial Logic:** Chess requires logic beyond simple data storage. Detecting check demands scanning the entire board for enemy attacks; validating a move demands temporarily simulating that move to ensure the King is not left exposed. This complexity tests real OOP design skills.
- **Universal Recognisability:** Chess rules are widely understood, making the project easy to demonstrate, test, and evaluate. Reviewers can immediately verify correctness by playing the game.

Beyond these academic reasons, building a chess engine from scratch provides genuine engineering experience: managing memory with dynamic polymorphism, designing a clean separation of concerns across multiple translation units, and implementing file I/O for game logging.

3. Purpose and Objectives

The project aims to achieve the following objectives:

1. Implement all six chess piece types with correct movement rules using an OOP class hierarchy.
2. Enforce legal move validation — a move is only permitted if it does not leave the moving player's King in check.
3. Detect check, checkmate, and stalemate game-ending conditions automatically.
4. Support special moves: castling (kingside and queenside) and pawn promotion.
5. Persist game history to a text file (move_history.txt) for post-game review.
6. Maintain a clean separation of concerns: piece logic (Pieces.cpp), board management (Board.cpp), display rendering (Display.cpp), and user interaction (main.cpp).
7. Provide a cross-platform CLI interface that compiles and runs on Windows, Linux, and macOS using standard C++17.

4. Problem Statement

Implementing a chess engine presents several non-trivial software engineering challenges:

Challenge 1 — Heterogeneous Piece Collection

A chess board holds up to 32 pieces of 6 different types simultaneously. Without polymorphism, handling each type distinctly would require massive conditional code. The solution must store all pieces through a uniform interface.

Challenge 2 — Move Legality vs. Move Validity

A piece's raw movement rule (e.g., a Rook moves in straight lines) is necessary but not sufficient. A move is only legal if, after the move is executed, the moving player's King is not in check. This requires a simulate-then-verify approach.

Challenge 3 — Checkmate and Stalemate Detection

Determining game termination requires enumerating every possible legal move for a player. If zero legal moves exist and the King is in check, it is checkmate; if zero legal moves exist and the King is safe, it is stalemate. This demands efficient enumeration of the entire moveable piece set.

Challenge 4 — Castling Logic

Castling is conditional on five simultaneous rules: neither the King nor the relevant Rook may have moved; all squares between them must be empty; the King must not currently be in check; and the King must not pass through or land on an attacked square.

Challenge 5 — Memory Management

Pieces are allocated on the heap via new to enable runtime polymorphism. Captured pieces must be deleted to avoid memory leaks. Pawn promotion replaces a Pawn object with a new higher-value piece.

5. Proposed Solution

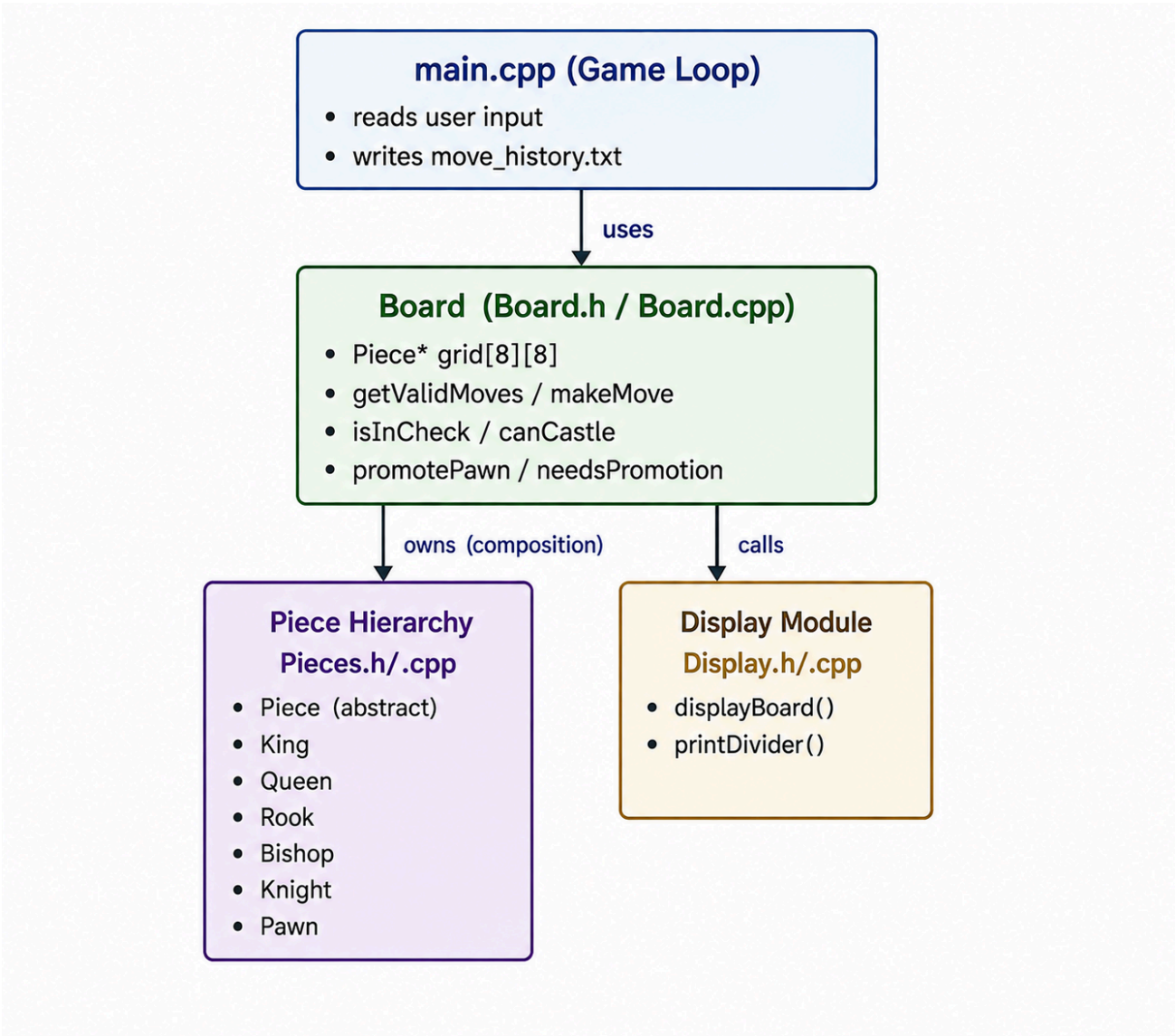
The solution is structured as a multi-file C++ project with a clear layered architecture:

Layer	Files
Piece Hierarchy	Pieces.h / Pieces.cpp
Board Management	Board.h / Board.cpp
Display / Rendering	Display.h / Display.cpp
Game Loop / CLI	main.cpp
Data Persistence	move_history.txt (runtime)

The Piece class is an abstract base class declaring a pure virtual isValidMove() method. Six concrete subclasses (King, Queen, Rook, Bishop, Knight, Pawn) inherit from Piece and implement their specific movement rules. The Board class owns a 8×8 grid of Piece* pointers, providing methods for move generation, legal move validation, check detection, and special moves. The Display module is entirely decoupled from game logic — it only reads the grid to render the board. main.cpp orchestrates the game loop: displaying the board, collecting user choices, executing moves, logging, and detecting termination.

6. System Architecture

6.1 Component Diagram



6.2 File Structure

File	Responsibility
Pieces.h	Declares abstract Piece base class and all 6 derived piece classes
Pieces.cpp	Implements movement validation logic for all 6 piece types

Board.h	Declares Board class with Position/MoveList structs and all public API
Board.cpp	Implements board initialisation, move generation, check detection, castling, promotion
Display.h	Declares displayBoard() and printDivider() free functions
Display.cpp	Implements ASCII board rendering; completely decoupled from game logic
main.cpp	Game loop: turn management, user I/O, file logging, termination detection
move_history.txt	Runtime log — records every move made during the current game session

7. OOP Concepts Applied

7.1 Classes and Objects

The project defines 8 classes in total: 1 abstract base class (Piece), 6 concrete piece subclasses (King, Queen, Rook, Bishop, Knight, Pawn), and the Board class. During a game, up to 33 objects are alive simultaneously — 32 Piece-derived objects on the board, plus the single Board instance.

```
// Class definition - Piece is an abstract class
class Piece {
protected:
    string identity;    // Piece type name
    string color;       // "White" or "Black"
    bool  hasMoved;     // Needed for castling / double pawn push
public:
    Piece(string identity, string color);
    virtual ~Piece();    // Virtual destructor for safe polymorphic
delete
    string getIdentity();
    string getColor();
    bool  getHasMoved();
    void  setHasMoved(bool moved);
    string getDisplay();
    virtual bool isValidMove(...) = 0; // Pure virtual - forces subclass
implementation
};
```

Objects are created on the heap to enable runtime polymorphism. The Board::init() method instantiates all 32 pieces, and the destructor cleans up every allocated object via delete.

7.2 Abstraction

Abstraction is achieved through the Piece abstract base class. The pure virtual method `isValidMove()` defines a contract: any concrete piece type must supply its own movement validation. Callers — such as `Board::getValidMoves()` — interact only with `Piece*`, never caring about the specific piece type.

```
// Board.cpp - calls isValidMove() without knowing the specific piece type
Piece* p = grid[r][c];
if (p->isValidMove(r, c, toR, toC, grid)) {    // Polymorphic dispatch
    // ...add to valid moves list
}

// Also used for check detection - same abstract interface:
if (grid[r][c]->isValidMove(r, c, king.r, king.c, grid))
    return true; // Enemy piece can attack the King
```

The `getDisplay()` virtual method provides a second layer of abstraction — it returns the board label for any piece (uppercase for White, lowercase for Black) without the caller needing to know the piece type.

7.3 Encapsulation

All Piece data members (identity, color, hasMoved) are declared protected, accessible only within the Piece hierarchy. External code accesses them exclusively through the public getter and setter methods. The Board class similarly encapsulates its 8×8 grid — external code cannot directly access `grid[][]` but must use `getPiece()`, `getValidMoves()`, `makeMove()`, etc.

```
class Piece {
protected:
    string identity;    // Not accessible outside Piece hierarchy
    string color;
    bool    hasMoved;
public:
    string getIdentity() { return identity; }    // Controlled read access
```



```
string getColor()    { return color; }
bool  getHasMoved() { return hasMoved; }
void  setHasMoved(bool moved) { hasMoved = moved; } // Controlled write
};

class Board {
    Piece* grid[8][8];    // Private - encapsulated grid
public:
    Piece* getPiece(int r, int c); // Controlled read access only
    // No setter for grid - changes only via makeMove()
};
```

7.4 Inheritance

The project uses single-level inheritance. All six piece types inherit from the Piece base class, reusing the common data members (identity, color, hasMoved) and basic getters while overriding isValidMove() with type-specific logic.

```
// Inheritance hierarchy
class King    : public Piece { public: King(string);    bool isValidMove(...)
override; };
class Queen   : public Piece { public: Queen(string);   bool isValidMove(...)
override; };
class Rook    : public Piece { public: Rook(string);    bool isValidMove(...)
override; };
class Bishop  : public Piece { public: Bishop(string);  bool isValidMove(...)
override; };
class Knight  : public Piece { public: Knight(string);  bool isValidMove(...)
override; };
class Pawn    : public Piece { public: Pawn(string);    bool isValidMove(...)
override; };

// Constructor reuse via base class initialiser list
King::King(string color) : Piece("King", color) {} // Calls Piece(identity,
color)
```

Each constructor delegates to the Piece base constructor via the member initialiser list, passing the piece name and colour. This eliminates code duplication — the common initialisation of identity, color, and hasMoved=false is written exactly once in Piece::Piece().

7.5 Polymorphism

Runtime (dynamic) polymorphism is the cornerstone of the design. Board stores Piece* pointers, and every call to isValidMove() dispatches to the correct subclass implementation at runtime through the virtual function table (vtable). This allows a single loop to validate moves for any piece type:

```
// Board::getValidMoves - one loop handles ALL 6 piece types
for (int toR = 0; toR < 8; toR++) {
    for (int toC = 0; toC < 8; toC++) {
        if (p->isValidMove(r, c, toR, toC, grid)) { // Virtual dispatch
            // Adds to move list for any piece type
        }
    }
}

// Board::isInCheck - same polymorphic call for attack detection
for (int r = 0; r < 8; r++)
    for (int c = 0; c < 8; c++)
        if (grid[r][c] && grid[r][c]->getColor() == enemy &&
            grid[r][c]->isValidMove(r, c, king.r, king.c, grid))
            return true; // Works for Knight, Rook, Bishop, Queen, etc.
```

The `getDisplay()` method also demonstrates polymorphism: when `displayBoard()` calls `p->getDisplay()` on each cell, it receives the correct piece label without branching on type. White pieces return uppercase labels (KING, PAWN) and Black pieces return lowercase (king, pawn).

7.6 Association — Aggregation and Composition

The Board class demonstrates composition with its `Piece*` grid. The Board creates all Piece objects in `init()` and destroys them in its destructor — meaning the Piece objects cannot exist independently of the Board, which is the defining characteristic of composition.

```
// Board owns its pieces - COMPOSITION
Board::Board() {
    for (int r = 0; r < 8; r++)
        for (int c = 0; c < 8; c++)
            grid[r][c] = nullptr;
    init(); // Creates all Piece objects
}

Board::~~Board() {
    for (int r = 0; r < 8; r++)
        for (int c = 0; c < 8; c++)
            delete grid[r][c]; // Board destroys all pieces on death
}

// Board::init() creates all Piece-derived objects:
grid[0][0] = new Rook("Black");
grid[0][4] = new King("Black");
// ... 32 pieces total
```

main.cpp holds a Board object by value on the stack. main uses Board via association (it calls its methods), but does not own individual pieces — that ownership belongs entirely to Board. The PieceList struct in main is a simple value struct that holds Position coordinates, not Piece pointers, further separating concerns.

7.7 Modular Design

Logic is cleanly separated across translation units following the Single Responsibility Principle. Each .h/.cpp pair has one clear purpose and minimal coupling:

Module	Single Responsibility
Pieces.h/.cpp	Defines what each piece IS and what moves are geometrically valid for it
Board.h/.cpp	Manages the board state — move generation, legal validation, check, castling, promotion
Display.h/.cpp	Renders the board to the terminal — has zero game logic
main.cpp	Drives the game session — turn management, user input, file logging, end detection

This separation means, for instance, that the display module could be replaced with a graphical renderer without touching any game logic. Equally, Board could be reused in an AI engine or network game without modification.

7.8 File Handling

Move history is persisted to move_history.txt using C++ file streams (fstream / ofstream). The file is created/cleared at game start with a header, then each move is appended during gameplay:

```
// main.cpp - initialise log at game start
ofstream initLog("move_history.txt");
if (initLog.is_open()) {
    initLog << "=== New Chess Game Started ===\n";
    initLog.close();
}

// Append each move during gameplay
ofstream logFile("move_history.txt", ios::app);
if (logFile.is_open()) {
```

```
    logFile << currentTurn << " " << selectedPiece->getIdentity()
        << " moved from " << board.getSquareName(selectedSq.r, selectedSq.c)
        << " to " << board.getSquareName(destination.r, destination.c)
        << "\n";
    logFile.close();
}

// Sample output in move_history.txt:
// === New Chess Game Started ===
// White Pawn moved from E2 to E4
// Black Pawn moved from E7 to E5
// White Knight moved from G1 to F3
```

8. Code Walkthrough & Snippets

8.1 Pieces.h — The Piece Hierarchy

The header declares the abstract Piece base class and all six derived types. The key design decision is the pure virtual isValidMove() signature, which receives a snapshot of the entire board (Piece* grid[8][8]) so each piece can check for blocking pieces along its path.

```
#ifndef PIECES_H
#define PIECES_H
#include <string>
using namespace std;

class Piece {
protected:
    string identity;
    string color;
    bool hasMoved;
public:
    Piece(string identity, string color);
    virtual ~Piece();
    string getIdentity();
    string getColor();
    bool getHasMoved();
    void setHasMoved(bool moved);
    // Pure virtual — each subclass MUST implement
    virtual bool isValidMove(int fromRow, int fromCol,
                           int toRow, int toCol,
                           Piece* grid[8][8]) = 0;
    virtual string getDisplay(); // Returns display label
};

class King : public Piece { public: King(string); bool isValidMove(...)
override; };
class Queen : public Piece { public: Queen(string); bool isValidMove(...)
override; };
class Rook : public Piece { public: Rook(string); bool isValidMove(...)
override; };
class Bishop : public Piece { public: Bishop(string); bool isValidMove(...)
override; };
class Knight : public Piece { public: Knight(string); bool isValidMove(...)
override; };
class Pawn : public Piece { public: Pawn(string); bool isValidMove(...)
override; };

#endif // PIECES_H
```

8.2 Board.h — Board Class Interface

Board.h declares the nested Position and MoveList structs (avoiding STL vectors for strict-array compliance), and the full public API of the Board class including castling and check-detection methods.

```
#ifndef BOARD_H
#define BOARD_H
#include "Pieces.h"
#include <string>

class Board {
    Piece* grid[8][8];    // Composition: Board owns all pieces
public:
    struct Position { int r, c; };
    struct MoveList { Position moves[32]; int count = 0; };

    Board();    // Calls init() — populates all 32 pieces
    ~Board();   // Deletes all allocated pieces

    void      init();
    void      display();
    Piece*    getPiece(int r, int c);
    string    getSquareName(int r, int c);    // e.g. (6,4) -> "E2"

    MoveList  getValidMoves(int r, int c);
    void      makeMove(int fromR, int fromC, int toR, int toC);

    bool      needsPromotion(int r, int c);
    void      promotePawn(int r, int c, int choice);

    Position  findKing(string color);
    bool      isInCheck(string color);
    bool      isLegalMove(int fromR, int fromC, int toR, int toC);
    bool      canCastle(string color, bool kingside);
};
#endif // BOARD_H
```

8.3 Pieces.cpp — Movement Logic for All 6 Pieces

Each piece implements a distinct movement algorithm. Below are annotated implementations of the most significant pieces.

King — One Step in Any Direction

```
bool King::isValidMove(int fromRow, int fromCol, int toRow, int toCol,
                       Piece* grid[8][8]) {
    int dR = abs(toRow - fromRow);
    int dC = abs(toCol - fromCol);
    if (dR > 1 || dC > 1) return false; // Max 1 step
    if (dR == 0 && dC == 0) return false; // Must actually move
    // Cannot capture own piece
    if (grid[toRow][toCol] != nullptr &&
        grid[toRow][toCol]->getColor() == color) return false;
    return true;
    // Note: does NOT check if destination is attacked -
    // that is handled by Board::isLegalMove().
}
```

Queen — Straight Lines + Diagonals with Path Checking

```
bool Queen::isValidMove(int fromRow, int fromCol, int toRow, int toCol,
                        Piece* grid[8][8]) {
    int dR = abs(toRow - fromRow);
    int dC = abs(toCol - fromCol);
    bool straight = (fromRow == toRow || fromCol == toCol);
    bool diagonal = (dR == dC);
    if (!straight && !diagonal) return false; // Must be a line

    // Step direction: -1, 0, or +1
    int stepR = (toRow > fromRow) - (toRow < fromRow);
    int stepC = (toCol > fromCol) - (toCol < fromCol);

    // Check all squares between source and destination
    int r = fromRow + stepR, c = fromCol + stepC;
    while (r != toRow || c != toCol) {
        if (grid[r][c] != nullptr) return false; // Blocked
        r += stepR; c += stepC;
    }
    if (grid[toRow][toCol] && grid[toRow][toCol]->getColor() == color)
        return false; // Cannot capture own piece
    return true;
}
```

Knight — L-Shape (No Path Checking)

```
bool Knight::isValidMove(int fromRow, int fromCol, int toRow, int toCol,
                          Piece* grid[8][8]) {
```

```
int dR = abs(toRow - fromRow);
int dC = abs(toCol - fromCol);
// Must be a 2+1 or 1+2 L-shape
bool lShape = (dR == 2 && dC == 1) || (dR == 1 && dC == 2);
if (!lShape) return false;
// Knights JUMP - no path check needed
if (grid[toRow][toCol] && grid[toRow][toCol]->getColor() == color)
    return false;
return true;
}
```

Pawn — Direction-Aware with Capture Logic

```
bool Pawn::isValidMove(int fromRow, int fromCol, int toRow, int toCol,
                       Piece* grid[8][8]) {
    int direction = (color == "White") ? -1 : 1; // White moves up (row-), Black
down
    int startRow = (color == "White") ? 6 : 1; // Starting row for double push
    int dR = toRow - fromRow;
    int dC = abs(toCol - fromCol);

    // Single forward step (no capture)
    if (dC == 0 && dR == direction && grid[toRow][toCol] == nullptr)
        return true;

    // Double push from starting rank
    if (dC == 0 && dR == 2*direction && fromRow == startRow &&
        grid[fromRow + direction][fromCol] == nullptr &&
        grid[toRow][toCol] == nullptr)
        return true;

    // Diagonal capture (must have an enemy piece)
    if (dC == 1 && dR == direction &&
        grid[toRow][toCol] != nullptr &&
        grid[toRow][toCol]->getColor() != color)
        return true;

    return false;
}
```


8.4 Board.cpp — Core Game Logic

Legal Move Validation via Simulation

The most important method in the project: `isLegalMove()` temporarily executes a move, checks whether it exposes the King, then undoes the move. This simulate-verify-undo pattern is essential for correct chess gameplay.

```
bool Board::isLegalMove(int fromR, int fromC, int toR, int toC) {
    Piece* moving    = grid[fromR][fromC];
    Piece* captured   = grid[toR][toC];    // Save the piece that may be captured
    string myColor    = moving->getColor();

    // Step 1: Temporarily execute the move
    grid[toR][toC]    = moving;
    grid[fromR][fromC] = nullptr;

    // Step 2: Check if own King is now in check
    bool safe = !isInCheck(myColor);

    // Step 3: Undo the move (MUST restore state)
    grid[fromR][fromC] = moving;
    grid[toR][toC]     = captured;

    return safe; // Move is legal only if King is not in check after it
}
```

Castling — Five-Rule Validation

```
bool Board::canCastle(string color, bool kingside) {
    int row = (color == "White") ? 7 : 0;
    int rookCol = kingside ? 7 : 0;

    // Rule 1: King exists and has not moved
    Piece* king = grid[row][4];
    if (!king || king->getIdentity() != "King" || king->getHasMoved())
        return false;

    // Rule 2: Rook exists and has not moved
    Piece* rook = grid[row][rookCol];
    if (!rook || rook->getIdentity() != "Rook" || rook->getHasMoved())
        return false;

    // Rule 3: All squares between King and Rook must be empty
    int start = (4 < rookCol) ? 5 : rookCol+1;
    int end   = (4 < rookCol) ? rookCol : 4;
    for (int c = start; c < end; c++)
        if (grid[row][c]) return false;

    // Rule 4: King must not be currently in check
```

```
if (isInCheck(color)) return false;

// Rule 5: King must not pass through or land on an attacked square
int passCol = kingside ? 5 : 3;
int landCol = kingside ? 6 : 2;
// Check pass-through square
grid[row][passCol] = king; grid[row][4] = nullptr;
bool passSafe = !isInCheck(color);
grid[row][4] = king; grid[row][passCol] = nullptr;
if (!passSafe) return false;
// Check landing square
grid[row][landCol] = king; grid[row][4] = nullptr;
bool landSafe = !isInCheck(color);
grid[row][4] = king; grid[row][landCol] = nullptr;
return landSafe;
}
```

Pawn Promotion

```
void Board::promotePawn(int r, int c, int choice) {
    string col = grid[r][c]->getColor();
    delete grid[r][c]; // Destroy the Pawn object
    switch (choice) {
        case 1: grid[r][c] = new Queen(col); break;
        case 2: grid[r][c] = new Rook(col); break;
        case 3: grid[r][c] = new Bishop(col); break;
        case 4: grid[r][c] = new Knight(col); break;
        default: grid[r][c] = new Queen(col); break; // Default to Queen
    }
}
```

8.5 Display.cpp — Board Rendering

The display module is entirely decoupled from game logic. It receives only the grid pointer and prints the board. Uppercase labels indicate White pieces; lowercase indicates Black pieces; dots indicate empty squares.

```
void displayBoard(Piece* grid[8][8]) {
    const int COL_W = 8;
    string cols[] = {" A ", " B ", " C ", " D ", " E ", " F ", " G ", " H "};

    cout << "\n BLACK\n\n";
    // Print column headers
    cout << "      ";
    for (int c = 0; c < 8; c++) cout << "||" << cols[c];
    cout << "||"; printDivider();

    for (int r = 0; r < 8; r++) {
```

```
    cout << "  " << (8 - r) << "  ";
    for (int c = 0; c < 8; c++) {
        cout << "||";
        if (grid[r][c]) {
            string label = grid[r][c]->getDisplay(); // Virtual call
            int pad = (COL_W - (int)label.size()) / 2;
            cout << string(pad, ' ') << label <<
string(pad+(COL_W-label.size())%2, ' ');
        } else {
            cout << "    "; // Empty square
        }
    }
    cout << "|| " << (8 - r) << "\n";
    printDivider();
}
}
```

8.6 main.cpp — Game Loop

The game loop implements turn management, movable piece enumeration, user selection with validation, move execution, file logging, check announcements, pawn promotion prompts, and termination detection.

```
int main() {
    Board board;
    string currentTurn = "White";
    // Clear log and write header
    ofstream initLog("move_history.txt");
    if (initLog.is_open()) { initLog << "=== New Chess Game Started ===\n";
initLog.close(); }

    while (true) {
        board.display();
        cout << "\n=== " << currentTurn << "'s Turn ===\n";

        // 1. Enumerate all movable pieces for the current player
        PieceList movablePieces;
        for (int r = 0; r < 8; r++) {
            for (int c = 0; c < 8; c++) {
                Piece* p = board.getPiece(r, c);
                if (p && p->getColor() == currentTurn) {
                    Board::MoveList moves = board.getValidMoves(r, c);
                    if (moves.count > 0) {
                        movablePieces.pieces[movablePieces.count++] = {r, c};
                    }
                }
            }
        }
    }
}
```

```
// 2. Detect game termination
if (movablePieces.count == 0) {
    if (board.isInCheck(currentTurn))
        cout << "\nCHECKMATE! " << (currentTurn=="White"? "Black": "White")
    << " wins!\n";
    else
        cout << "\nSTALEMATE! The game is a draw.\n";
    break;
}

// 3. Present piece selection menu and move selection menu
// ...user input handling and validation...

// 4. Execute move and log it
board.makeMove(selectedSq.r, selectedSq.c, destination.r, destination.c);
// ...file logging...

// 5. Announce check / handle promotion / alternate turns
}
return 0;
}
```

9. How to Build and Run

The project uses only the C++ Standard Library and compiles with any C++11 (or later) conforming compiler. No third-party libraries are required.

9.1 Prerequisites

Platform	Compiler / Command
Windows	g++ via MinGW/MSYS2 OR MSVC via Visual Studio OR cl.exe (command line)
Linux / macOS	g++ (GNU Compiler Collection) or clang++

9.2 Compilation Commands

Windows — MinGW / MSYS2

```
g++ -std=c++17 -o chess.exe main.cpp Board.cpp Pieces.cpp Display.cpp
chess.exe
```

Linux / macOS — g++ or clang++

```
g++ -std=c++17 -o chess main.cpp Board.cpp Pieces.cpp Display.cpp
./chess

# Or with clang++:
clang++ -std=c++17 -o chess main.cpp Board.cpp Pieces.cpp Display.cpp
./chess
```

Windows — Visual Studio (cl.exe)

```
cl /EHsc /std:c++17 main.cpp Board.cpp Pieces.cpp Display.cpp /Fe:chess.exe
chess.exe
```

9.3 After Running

On game start, a `move_history.txt` file is created (or overwritten) in the same directory as the executable. Every move is appended to this file during the game. The file persists after the game ends and can be opened in any text editor for review.

Note on `system("pause")`

`main.cpp` includes `system("pause")` at exit for convenience on Windows (keeps the terminal window open). On Linux/macOS this command does nothing harmful but may print an error message. Remove or replace it with `cin.get()` for fully cross-platform behaviour.

10. Sample Gameplay Output

Below is a representative terminal session showing the board display format, turn prompts, move selection, check announcement, and pawn promotion prompt.

```

BLACK

  || A || B || C || D || E || F || G || H ||
++-----++
8 || rook || knight || bishop || queen || king || bishop || knight || rook || 8
++-----++
7 || pawn || pawn || pawn || pawn || pawn || pawn || pawn || pawn || 7
++-----++
6 || .... || .... || .... || .... || .... || .... || .... || .... || 6
++-----++
5 || .... || .... || .... || .... || .... || .... || .... || .... || 5
++-----++
4 || .... || .... || .... || .... || .... || .... || .... || .... || 4
++-----++
3 || .... || .... || .... || .... || .... || .... || .... || .... || 3
++-----++
2 || PAWN || PAWN || PAWN || PAWN || PAWN || PAWN || PAWN || PAWN || 2
++-----++
1 || ROOK || KNIGHT || BISHOP || QUEEN || KING || BISHOP || KNIGHT || ROOK || 1
++-----++
  || A || B || C || D || E || F || G || H ||

WHITE

UPPERCASE = White pieces
lowercase = Black pieces
....      = Empty square

=== White's Turn ===
Which piece would you like to move?
Press 1 for Pawn at A2
Press 2 for Pawn at B2
Press 3 for Pawn at C2
Press 4 for Pawn at D2
Press 5 for Pawn at E2
Press 6 for Pawn at F2
Press 7 for Pawn at G2
Press 8 for Pawn at H2
Press 9 for Knight at B1
Press 10 for Knight at G1
Your choice: 5

Where do you want to move the Pawn at E2?
Press 1 for E4
Press 2 for E3
Press 0 to go back and pick a different piece.
Your choice: 2

```

```

      BLACK
      || A || B || C || D || E || F || G || H ||
      +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+
8 || rook || knight || bishop || queen || king || bishop || knight || rook || 8
      +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+
7 || pawn || pawn || pawn || pawn || pawn || pawn || pawn || pawn || 7
      +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+
6 || .... || .... || .... || .... || .... || .... || .... || .... || 6
      +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+
5 || .... || .... || .... || .... || .... || .... || .... || .... || 5
      +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+
4 || .... || .... || .... || .... || .... || .... || .... || .... || 4
      +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+
3 || .... || .... || .... || .... || PAWN || .... || .... || .... || 3
      +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+
2 || PAWN || PAWN || PAWN || PAWN || .... || PAWN || PAWN || PAWN || 2
      +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+
1 || ROOK || KNIGHT || BISHOP || QUEEN || KING || BISHOP || KNIGHT || ROOK || 1
      +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+
      || A || B || C || D || E || F || G || H ||

      WHITE

UPPERCASE = White pieces
lowercase = Black pieces
....      = Empty square

=== Black's Turn ===
Which piece would you like to move?
Press 1 for Knight at B8
Press 2 for Knight at G8
Press 3 for Pawn at A7
Press 4 for Pawn at B7
Press 5 for Pawn at C7
Press 6 for Pawn at D7
Press 7 for Pawn at E7
Press 8 for Pawn at F7
Press 9 for Pawn at G7
Press 10 for Pawn at H7
Your choice: █

```

11. Conclusion

This project successfully demonstrates that OOP principles are not merely theoretical constructs — they are practical tools that make complex software easier to design, extend, and maintain. Chess, with its six distinct piece types, naturally demanded a class hierarchy; its rich gameplay rules drove the need for clean separation between movement validation, board state management, and display rendering.

The key learning outcomes from this project are: polymorphism eliminates type-switching code and makes the board engine extensible (adding a new piece type requires only a new subclass); encapsulation prevents unintended state modification and makes the Board class a reliable, self-contained service; and modular design dramatically simplifies testing — the Display module was verified independently of the game logic.

The project goes beyond the minimum requirements by implementing castling, pawn promotion, stalemate detection, full input validation with error recovery, and persistent move logging to a file. These features collectively demonstrate the Complexity, Design Quality, and Depth criteria outlined in the assignment rubric.

Repository

<https://github.com/Haider-3100/CS-112-Project> All source files: main.cpp, Board.h/.cpp, Pieces.h/.cpp, Display.h/.cpp, move_history.txt, chess.exe

End of Report — CS112 Chess Engine Project